ENSET Mohammedia — École Normale Supérieure de l'Enseignement Technique

Master SDIA — Systèmes Distribués et Intelligence Artificielle

Mini-Projet

Recherche Heuristique, A* et Chaînes de Markov

Étudiant	**Felouach Oussama**
Encadrant	**Prof. Mohammed Mestari**
Filière	Systèmes Distribués et Intelligence Artificielle
Module	Bases de l'Intelligence Artificielle
Date	Mars 2026

Année universitaire 2025–2026

Contents

1 Introduction **2**

2 Fondements theoriques **2**
2.1 Formalisation du probleme de recherche sur graphe . . . 2
2.2 Role des structures OPEN et CLOSED . . . 2
2.3 Fonctions d'evaluation et comparaison des algorithmes . . . 3
2.4 Admissibilite et coherence des heuristiques . . . 3
2.5 Weighted A* et compromis optimalite/rapidite . . . 4

3 Etude analytique – Graphe de reference **4**
3.1 Presentation du graphe G_A . . . 4
3.2 Trace pas a pas – Greedy Best-First Search . . . 4
3.3 Trace pas a pas – A* . . . 5
3.4 Comparaison et analyse critique . . . 5
3.5 Trois familles d'heuristiques – impact sur A* . . . 5

4 Implementation Python **6**
4.1 Architecture logicielle . . . 6
4.2 Algorithme A* generique . . . 6
4.3 Analyse de Markov – extrait . . . 6

5 Analyse experimentale **7**
5.1 Complexite theorique . . . 7
5.2 Extension E3 – Weighted A* : compromis cout/expansions . . . 7
5.3 Etude de scalabilite sur graphes aleatoires . . . 7
5.4 Extension E1 – Dominance heuristique . . . 8

6 Chaine de Markov absorbante – Ruine du Joueur **9**
6.1 Modelisation du probleme . . . 9
6.2 Matrice de transition . . . 9
6.3 Forme canonique . . . 10
6.4 Matrice fondamentale . . . 10
6.5 Probabilites d'absorption et esperance du temps . . . 11
6.6 Validation Monte Carlo . . . 11
6.7 Extensions – cas asymetrique et sensibilite . . . 11

7 Discussion critique et Conclusion **13**
7.1 Forces, limites et pistes d'amelioration . . . 13
7.2 Conclusion . . . 13

1. Introduction

La capacité d'un système autonome à naviguer efficacement dans un espace d'états complexe est au cœur de l'intelligence artificielle appliquée. Ce mini-projet traite deux dimensions complémentaires de cette problématique :

- **Dimension déterministe** : comment des stratégies de recherche informée (Greedy Best-First, A*, Weighted A*) et non informée (UCS) orientent-elles l'exploration d'un graphe orienté pondéré tout en garantissant, selon le cas, rapidité et optimalité ?
- **Dimension stochastique** : comment modéliser et analyser rigoureusement l'évolution d'un système aléatoire jusqu'à son absorption, via la théorie des chaînes de Markov absorbantes appliquée au problème classique de la ruine du joueur ?

Problématique centrale : *Comment concevoir un système d'exploration intelligent garantissant efficacité et optimalité, tout en modélisant formellement l'incertitude par un cadre probabiliste rigoureux ?*

Le rapport est structuré en huit parties : fondements théoriques, étude analytique du graphe, implémentation Python, analyse expérimentale, étude Markov, extensions, discussion critique et conclusion.

2. Cadre Théorique

2.1. Formalisation d'un problème de recherche sur graphe

Un problème de recherche est défini par le quintuplet $\langle S, s_0, G, T, c\rangle$ où :

- S : ensemble fini d'états
- $s_0 \in S$: état initial
- $G \subseteq S$: ensemble des états but (test de but : $\mathrm{GOAL}(n)$)
- $T : S \to 2^S$: fonction de transition (successeurs)
- $c : S \times S \to \mathbb{R}^+$: coût de transition

Les fonctions d'évaluation :

$$g(n) = \text{coût cumulé depuis } s_0 \qquad h(n) = \text{estimation heuristique du coût restant} \qquad f(n) = g(n)+h(n)$$

2.2. Rôle des structures OPEN et CLOSED

OPEN (file de priorité) contient les nœuds générés mais non encore développés. **CLOSED** (ensemble de hachage) contient les nœuds déjà développés.

Le problème des **boucles** et de la **redondance** : un même état peut être atteint par plusieurs chemins. Sans CLOSED, l'algorithme peut boucler indéfiniment. La mise à jour consiste à remplacer un nœud dans OPEN si un meilleur g est trouvé. La **réouverture** d'un nœud de CLOSED est nécessaire quand h n'est pas cohérente — un nœud clos peut recevoir un meilleur coût via un autre chemin.

2.3. Fonctions d'évaluation des algorithmes

$$f_{\mathrm{UCS}}(n) = g(n) \qquad f_{\mathrm{Greedy}}(n) = h(n) \qquad f_{A^*}(n) = g(n) + h(n)$$

UCS explore par coût croissant sans information — exhaustif mais optimal. Greedy exploite uniquement h — rapide mais non optimal. A* combine les deux, offrant optimalité sous conditions d'admissibilité.

Mise a jour de OPEN. Si un noeud $n \in$ OPEN est atteint avec $g'(n) < g(n)$, sa priorite doit etre mise a jour. En pratique, on utilise la *lazy deletion* : on insere une nouvelle entree dans le tas et on ignore les entrees obsoletes lors de l'extraction.

2.3 Fonctions d'evaluation et comparaison des algorithmes

Les trois strategies se distinguent uniquement par leur fonction d'evaluation :

Algorithme	$f(n)$	**Guide par**	**Optimal ?**	**Complet ?**
UCS	$g(n)$	Cout reel uniquement	Oui (si $c > 0$)	Oui
Greedy BFS	$h(n)$	Heuristique uniquement	Non en general	Oui (fini)
A*	$g(n) + h(n)$	Cout reel + heuristique	Oui (h admissible)	Oui

- **UCS** est aveugle a l'heuristique : il garantit l'optimalite mais explore radialement sans guidance, similaire a l'algorithme de Dijkstra.
- **Greedy BFS** est myope : il fonce vers ce qui semble le plus pres du but sans tenir compte des couts accumules, ce qui peut le mener a des impasses ou des chemins sous-optimaux.
- **A*** realise le meilleur compromis : en combinant g (ce que l'on sait deja) et h (ce que l'on estime), il dirige la recherche tout en preservant la garantie d'optimalite.

2.4 Admissibilite et coherence des heuristiques

Définition 2.2 (Admissibilite)**.** Une heuristique h est **admissible** si pour tout etat n :

$$h(n) \leq h^*(n)$$

ou $h^*(n)$ est le cout reel optimal de n vers le but. Une heuristique admissible ne surestime jamais le cout restant.

Définition 2.3 (Coherence (monotonie))**.** Une heuristique h est **coherente** si pour tout arc (n, n') de cout $c(n, n')$:

$$h(n) \leq c(n, n') + h(n').$$

Il s'agit de l'inegalite triangulaire appliquee a l'heuristique. Toute heuristique coherente est admissible. La reciproque est fausse.

Propriété 2.1. Avec h **admissible**, A* est optimal en recherche sur arbre. Avec h **coherente**, A* est optimal en recherche sur graphe sans re-ouverture des noeuds de CLOSED, et les valeurs f sont non decroissantes le long de tout chemin explore.

Impact pratique.

- Heuristique *admissible et coherente* : implementation la plus simple et la plus efficace — aucun noeud de CLOSED n'a besoin d'etre re-ouvert.
- Heuristique *admissible mais non coherente* : A* doit autoriser la re-ouverture de CLOSED pour garantir l'optimalite, au prix d'une implementation plus complexe.
- Heuristique *non admissible* : A* peut trouver une solution sous-optimale, parfois en explorant moins de noeuds (utile si l'on accepte une approximation).

2.5 Weighted A* et compromis optimalite/rapidite

Définition 2.4 (Weighted A*)**.** La fonction d'evaluation de Weighted A* est :

$$f_w(n) = g(n) + w \cdot h(n), \quad w \geq 1.$$

Propriété 2.2 (Garantie sous-optimale bornee)**.** Si h est admissible, Weighted A* garantit que le cout de la solution retournee satisfait cout $\leq w \cdot C^*$, ou C^* est le cout optimal.

En pratique : $w = 1$ correspond a A* standard (optimal), $w \to \infty$ degenere en Greedy BFS. Pour $w \in (1, +\infty)$, on obtient une solution dont l'ecart a l'optimal est controle par w, avec un nombre d'expansions generalement reduit.

3 Etude analytique – Graphe de reference

3.1 Presentation du graphe G_A

Le graphe oriente pondere G_A (etat initial A, but G) est defini par les arcs du tableau ci-dessous. La heuristique h_1, admissible et coherente, est donnee pour chaque noeud.

Arc	Cout	Noeud	h_1
$A \to B$	1	A	6
$A \to C$	4	B	5
$B \to C$	2	C	4
$B \to D$	5	D	2
$C \to D$	1	E	4
$C \to E$	3	F	1
$D \to G$	2	G	0
$E \to G$	4		
$F \to G$	1		

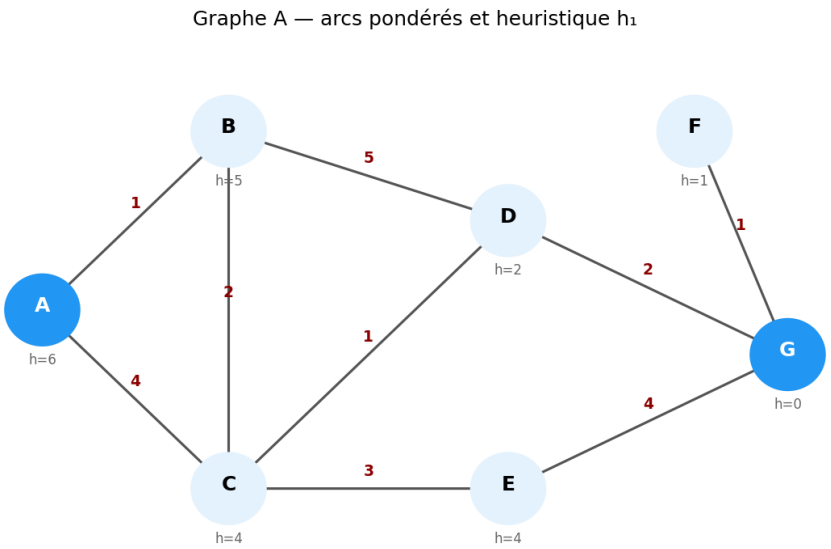

Figure 1: Graphe G_A : couts d'arcs (rouge) et valeurs heuristiques h_1 (gris sous les noeuds)

3.2 Trace pas a pas – Greedy Best-First Search

GBFS extrait de OPEN le noeud de plus petite valeur h, sans tenir compte de g.

Etape	Noeud extrait	g	h	f=h	OPEN apres expansion
0	A	0	6	6	$\{B(h=5),\ C(h=4)\}$
1	C	4	4	4	$\{B(5),\ D(2),\ E(4)\}$
2	D	5	2	2	$\{B(5),\ E(4),\ G(0)\}$
3	**G**	7	0	0	—

Resultat : chemin $A \to C \to D \to G$, cout $= 4 + 1 + 2 =$ **7** (sous-optimal).

3.3 Trace pas a pas – A*

A* extrait de OPEN le noeud de plus petite valeur $f = g + h$.

Etape	Noeud extrait	g	h	f=g+h	OPEN apres expansion
0	A	0	6	6	$\{B(f=6),\ C(f=8)\}$
1	B	1	5	6	$\{C(f=7),\ D(f=8)\}$
2	C	3	4	7	$\{D(f=6),\ E(f=10)\}$
3	D	4	2	6	$\{G(f=6),\ E(f=10)\}$
4	**G**	6	0	6	—

Resultat : chemin $A \to B \to C \to D \to G$, cout $= 1 + 2 + 1 + 2 =$ **6** (optimal).

3.4 Comparaison et analyse critique

Algorithme	Chemin trouve	Cout	Optimal ?	Expansions
Greedy BFS	$A \to C \to D \to G$	7	Non	4
A*	$A \to B \to C \to D \to G$	6	Oui	5
UCS	$A \to B \to C \to D \to G$	6	Oui	6

Analyse de la sous-optimalite de GBFS. A l'etape 0, GBFS choisit C plutot que B car $h(C) = 4 < h(B) = 5$, en ignorant que $g(C) = 4 \gg g(B) = 1$. Ce choix myope conduit a un chemin total de cout 7 au lieu de 6. GBFS n'a aucune garantie sur la qualite de la solution : il peut trouver l'optimal par chance, mais peut aussi echouer meme sur des graphes simples.

Pourquoi A* est-il optimal ? Sous l'admissibilite de h_1, A* satisfait l'invariant suivant : lorsque le but est extrait de OPEN, g(but) est le cout minimal possible. En effet, $f(n) = g(n) + h(n) \leq g(n) + h^*(n)$, ce qui signifie que $f(n)$ est un minorant du cout de tout chemin passant par n. Aucun chemin non encore explore ne peut donc etre moins couteux que le chemin optimal trouve.

3.5 Trois familles d'heuristiques – impact sur A*

Heuristique	Type	A	B	C	D	E	F	G
h_1	Admissible + coherente	6	5	4	2	4	1	0
h_2	Admissible, non coherente	6	5	5	2	4	1	0
h_3	Non admissible	8	7	7	5	6	3	0

Non-coherence de h_2 : Pour l'arc $C \to D$ de cout 1 : $h_2(C) = 5 > c(C, D) + h_2(D) = 1 + 2 = 3$. L'inegalite triangulaire est violee. Cependant, $h_2(n) \leq h^*(n)$ pour tout n, donc h_2

reste admissible et A* retourne une solution optimale (a condition d'autoriser la re-ouverture de CLOSED).

Non-admissibilite de h_3 : $h_3(A) = 8 > h^*(A) = 6$. En surestimant le cout restant depuis A, A* peut ecarter prematurement le chemin optimal. Sur ce graphe, la solution reste optimale car l'overestimation est localisee, mais ce n'est pas garanti en general.

4 Implementation Python

4.1 Architecture logicielle

Le projet adopte une architecture modulaire en cinq paquets : `search/` (algorithmes de recherche), `markov/` (analyse analytique + simulation), `experiments/` (benchmarks et graphiques), et `main.py` (point d'entree). Chaque expansion est enregistree par un `SearchLogger` (noeud, g, h, f, taille frontiere, temps), garantissant la reproductibilite.

4.2 Algorithme A* generique

L'implementation unifiee couvre UCS ($w_g = 1, w_h = 0$), Greedy ($w_g = 0, w_h = 1$), A* ($w_g = w_h = 1$) et Weighted A* ($w_h = w$).

Listing 1: search/astar.py – noyau unifie UCS/Greedy/A*/Weighted A*

```python
def search(graph, start, goal, heuristic, w_g=1.0, w_h=1.0, allow_reopen=False):
    g_scores = {start: 0.0};  came_from = {start: None}
    OPEN = [];  CLOSED = set();  counter = 0
    heapq.heappush(OPEN, (w_h * heuristic(start, goal), 0, 0.0, start))
    while OPEN:
        f, _, g, node = heapq.heappop(OPEN)
        if node in CLOSED: continue
        CLOSED.add(node)
        if node == goal: return reconstruct_path(came_from, goal)
        for nbr, cost in graph.get(node, []):
            tg = g + cost
            if tg < g_scores.get(nbr, float('inf')):
                g_scores[nbr] = tg;  came_from[nbr] = node
                heapq.heappush(OPEN, (w_g*tg + w_h*heuristic(nbr,goal), counter, tg, nbr))
                counter += 1
                if allow_reopen: CLOSED.discard(nbr)
```

Points cles : *lazy deletion* (evite la mise a jour du tas en $O(n)$) ; *tie-breaking* par compteur deterministe ; re-ouverture de CLOSED activable pour les heuristiques non coherentes.

4.3 Analyse de Markov – extrait

La matrice fondamentale et les probabilites d'absorption sont calculees analytiquement : $N = (I - Q)^{-1}$, $B = NR$, $\mathbf{t} = N\mathbf{1}$, en $O(n^3)$ via `numpy.linalg.inv`.

5 Analyse experimentale

5.1 Complexite theorique

Algorithme	Temps	Espace	Remarque
UCS	$O(b^{C^*/\varepsilon})$	$O(b^{C^*/\varepsilon})$	b = facteur de branchement, ε = cout min d'arc
Greedy BFS	$O(b^m)$ pire cas	$O(b^m)$	m = profondeur max. Efficace en pratique
A*	$O(b^d)$	$O(b^d)$	d = profondeur solution ; avec h admissible
Weighted A*	$O(b^{d/w})$ typ.	idem	Reduit les expansions, perd la garantie optimale

5.2 Extension E3 – Weighted A* : compromis cout/expansions

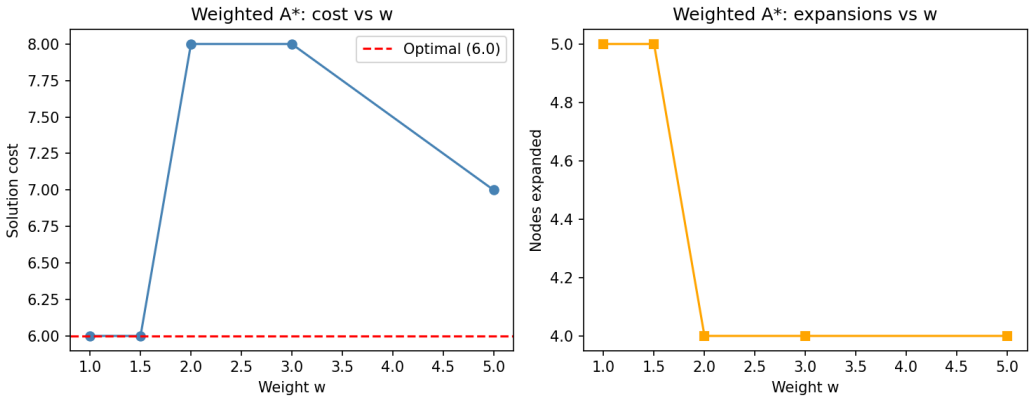

Figure 2: Weighted A* : cout de la solution et nombre d'expansions en fonction de w

w	Chemin	Cout	Expansions
1.0	$A \to B \to C \to D \to G$	6 (optimal)	5
1.5	$A \to B \to C \to D \to G$	6 (optimal)	5
2.0	$A \to B \to D \to G$	8 (+33%)	4
3.0	$A \to B \to D \to G$	8 (+33%)	4
5.0	$A \to C \to D \to G$	7 (+17%)	4

A partir de $w = 2$, A* ne garantit plus l'optimalite mais reduit d'une expansion. Sur ce petit graphe l'economie est marginale ; sur des graphes de taille industrielle (milliers de noeuds), la reduction du nombre d'expansions devient decisive.

5.3 Etude de scalabilite sur graphes aleatoires

Des graphes aleatoires diriges ($n \in \{10, 20, 50, 100, 200\}$ noeuds, $3n$ arcs, couts uniformes dans $[1, 10]$) ont ete generes avec un generateur reproductible. La heuristique utilisee est $h(i) = |goal - i|$, admissible pour les graphes acycliques croissants.

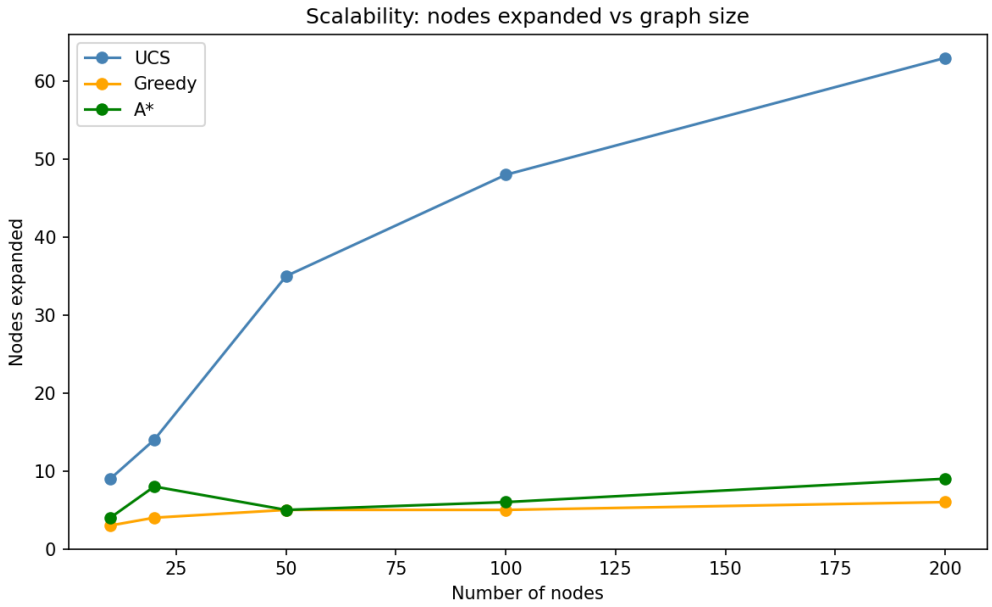

Figure 3: Nombre de noeuds developpes en fonction de la taille du graphe

Noeuds	UCS	Greedy	A*
10	9	3	4
20	14	4	8
50	35	5	5
100	48	5	6
200	63	6	9

UCS croit lineairement avec n car il explore radialement sans guidance. Greedy BFS reste quasi-constant grace a son guidage agressif, mais sans garantie optimale. A* se positionne entre les deux, profitant de l'heuristique tout en verifiant les couts reels.

5.4 Extension E1 – Dominance heuristique

Définition 5.1 (Dominance). h_2 **domine** h_1 si $h_2(n) \geq h_1(n)$ pour tout n, les deux etant admissibles. A* avec h_2 developpe au plus autant de noeuds qu'avec h_1.

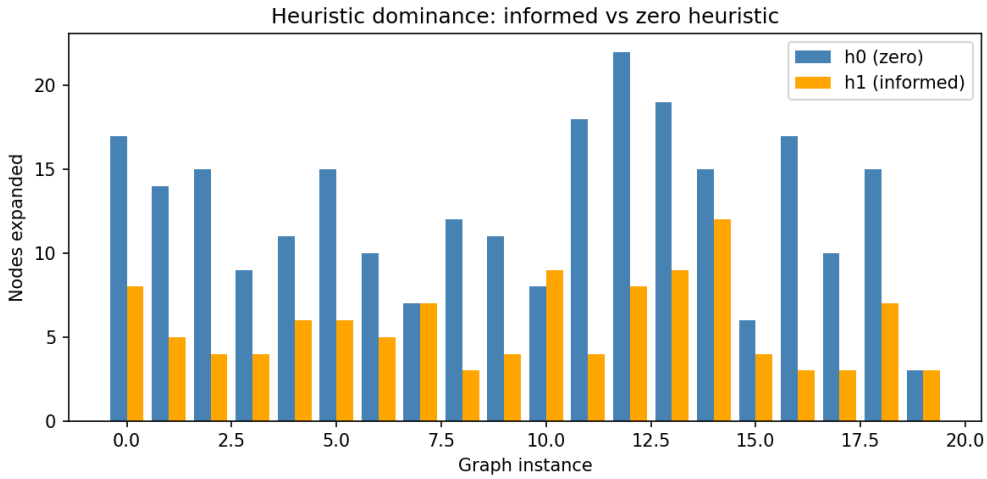

Figure 4: Extension E1 : dominance heuristique sur 20 graphes aleatoires (30 noeuds, 80 arcs)

Sur 20 graphes aleatoires, la heuristique informee ($h = |goal - i|$) est strictement meilleure que $h_0 \equiv 0$ dans **17 cas sur 20** (moyenne : 5.7 expansions vs 12.7), soit une reduction de 55%. Les 3 cas ou Greedy est equivalent correspondent a des graphes tres denses ou le chemin optimal est presque direct.

6 Chaine de Markov absorbante – Ruine du Joueur

6.1 Modelisation du probleme

On considere un joueur dont la fortune evolue sur l'espace d'etats $S = \{0, 1, 2, 3, 4, 5, 6\}$. A chaque etape independante, il gagne 1 unite monetaire avec probabilite p ou en perd 1 avec probabilite $q = 1 - p$. Les etats 0 (ruine totale) et 6 (victoire) sont **absorbants** : une fois atteints, le processus s'y maintient indefiniment.

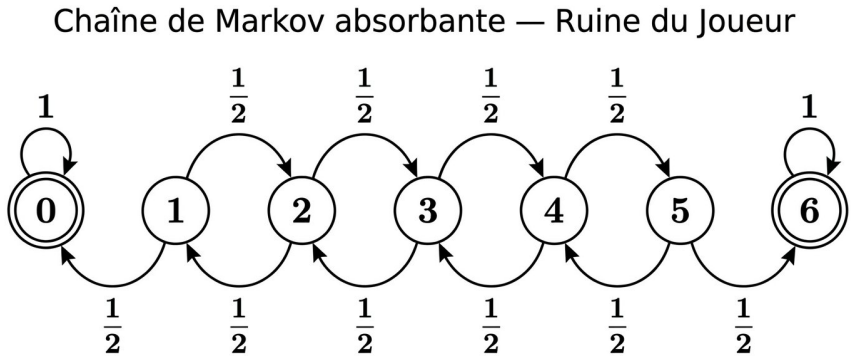

Figure 5: Chaine de Markov absorbante – Ruine du Joueur. Les etats 0 et 6 sont absorbants.

6.2 Matrice de transition

Pour $p = q = 0.5$ (jeu equitable), la matrice de transition P de taille 7×7 (etats ordonnes $0, 1, \ldots, 6$) vaut :

$$P = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}$$

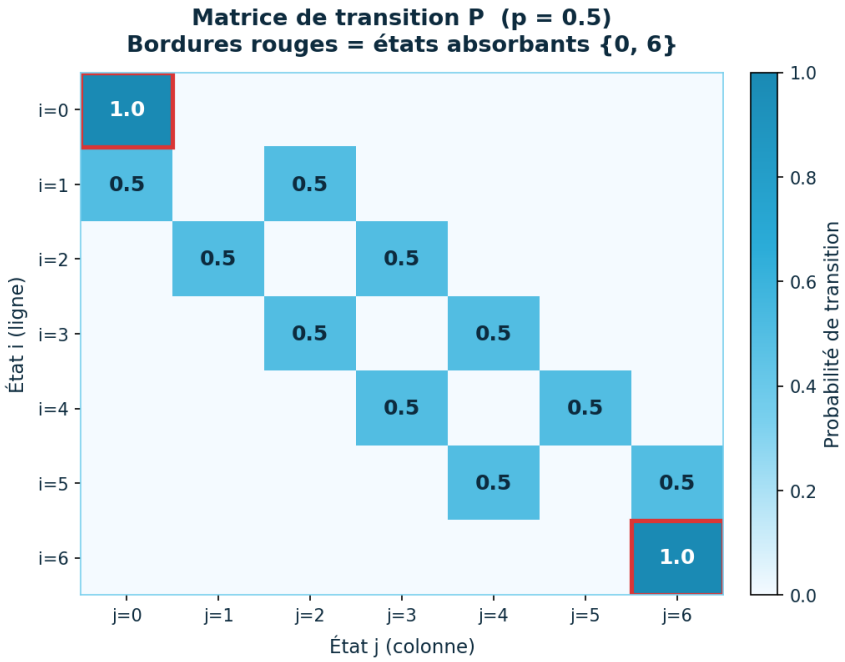

Figure 6: Visualisation de la matrice de transition P ($p = 0.5$). La structure tridiagonale des etats transitoires et les boucles des etats absorbants (encadres en rouge) sont clairement visibles.

6.3 Forme canonique

En reordonnant les etats (absorbants $\{0, 6\}$ en premier, transitoires $\{1, 2, 3, 4, 5\}$ ensuite) :

$$P_{\text{canon}} = \left(\begin{array}{c|c} I_{2\times 2} & \mathbf{0} \\ \hline R & Q \end{array}\right)$$

avec les sous-matrices :

$$Q = \frac{1}{2}\begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \qquad R = \frac{1}{2}\begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}$$

Q est la sous-matrice transitoire→transitoire et R la sous-matrice transitoire→absorbant.

6.4 Matrice fondamentale

Définition 6.1 (Matrice fondamentale)**.**

$$N = (I - Q)^{-1}$$

N_{ij} est le nombre espere de visites a l'etat transitoire j avant absorption, en partant de l'etat i.

Pour $p = 0.5$:

$$N = \begin{pmatrix} \frac{5}{3} & \frac{4}{3} & 1 & \frac{2}{3} & \frac{1}{3} \\ \frac{4}{3} & \frac{8}{3} & 2 & \frac{4}{3} & \frac{2}{3} \\ 1 & 2 & 3 & 2 & 1 \\ \frac{2}{3} & \frac{4}{3} & 2 & \frac{8}{3} & \frac{4}{3} \\ \frac{1}{3} & \frac{2}{3} & 1 & \frac{4}{3} & \frac{5}{3} \end{pmatrix}$$

6.5 Probabilites d'absorption et esperance du temps

$$B = NR \Rightarrow B = \begin{pmatrix} \frac{5}{6} & \frac{1}{6} \\ \frac{4}{6} & \frac{2}{6} \\ \frac{3}{6} & \frac{3}{6} \\ \frac{2}{6} & \frac{4}{6} \\ \frac{1}{6} & \frac{5}{6} \end{pmatrix} \qquad \mathbf{t} = N\mathbf{1} \Rightarrow$$

i	$E[T \mid i]$	$P(\text{ruine} \mid i)$
1	5.00	5/6
2	8.00	4/6
3	9.00	3/6
4	8.00	2/6
5	5.00	1/6

Resultats cles pour l'etat initial $i = 2$ (p=0.5) :

$$P(\text{ruine} \mid 2) = \frac{2}{3} \approx 0.6667 \qquad P(\text{victoire} \mid 2) = \frac{1}{3} \approx 0.3333 \qquad E[T \mid 2] = 8 \text{ etapes}$$

Formule analytique exacte : $P(\text{ruine} \mid i) = 1 - i/N = 1 - 2/6$ et $E[T \mid i] = i(N - i) = 2 \times 4 = 8$.

6.6 Validation Monte Carlo

200 000 simulations independantes (seed fixe) confirment les resultats analytiques avec un ecart maximal de 0.2% :

Quantite	Analytique	Monte Carlo	$\lvert\Delta\rvert$
$P(\text{ruine} \mid 2)$	0.666667	0.667630	0.0010
$P(\text{victoire} \mid 2)$	0.333333	0.332370	0.0010
$E[T \mid 2]$	8.0000	7.9867	0.0133

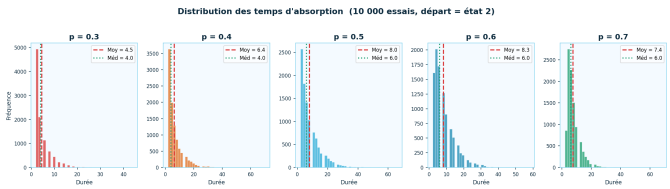

Figure 7: Distribution des temps d'absorption ($p \in \{0.3, \ldots, 0.7\}$, 10 000 essais, depart=2).

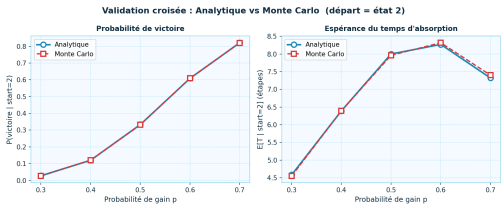

Figure 8: Validation croisee analytique vs Monte Carlo. Accord $< 1\%$ sur tous les p.

6.7 Extensions – cas asymetrique et sensibilite

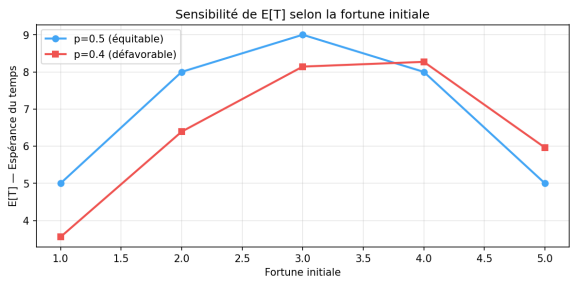

Figure 9: Sensibilite de $E[T \mid i]$ pour $p = 0.5$ et $p = 0.4$

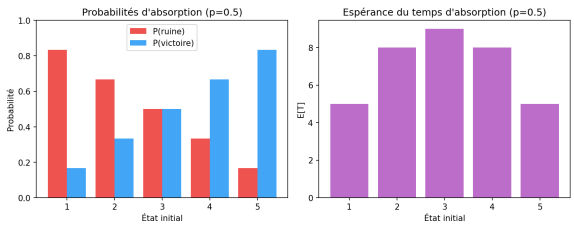

Figure 10: Probabilites d'absorption et $E[T]$ par etat ($p = 0.5$)

Pour $p = 0.4$ (jeu defavorable), la probabilite de ruine depuis l'etat 2 passe de 66.7% a **87.97%** ; Monte Carlo confirme ($\Delta = 0.0006$). $E[T]$ atteint son maximum en $i = 3$ pour $p = 0.5$ et se

deplace vers $i = 4$ pour $p = 0.4$: le biais vers la ruine allonge le parcours avant absorption dans l'etat 0.

	Cas symetrique ($p = 0.5$)			**Cas asymetrique** ($p = 0.4$)		
Etat	P(ruine)	P(vic.)	$E[T]$	P(ruine)	P(vic.)	$E[T]$
1	0.833	0.167	5.00	0.952	0.048	3.56
2	0.667	0.333	8.00	0.880	0.120	6.39
3	0.500	0.500	9.00	0.771	0.229	8.14
4	0.333	0.667	8.00	0.609	0.391	8.27
5	0.167	0.833	5.00	0.365	0.635	5.96

Extension E5 – Sensibilite selon la fortune initiale k

La figure 11 generalise l'analyse a trois valeurs de p et tous les etats transitoires $k \in \{1, \ldots, 5\}$.

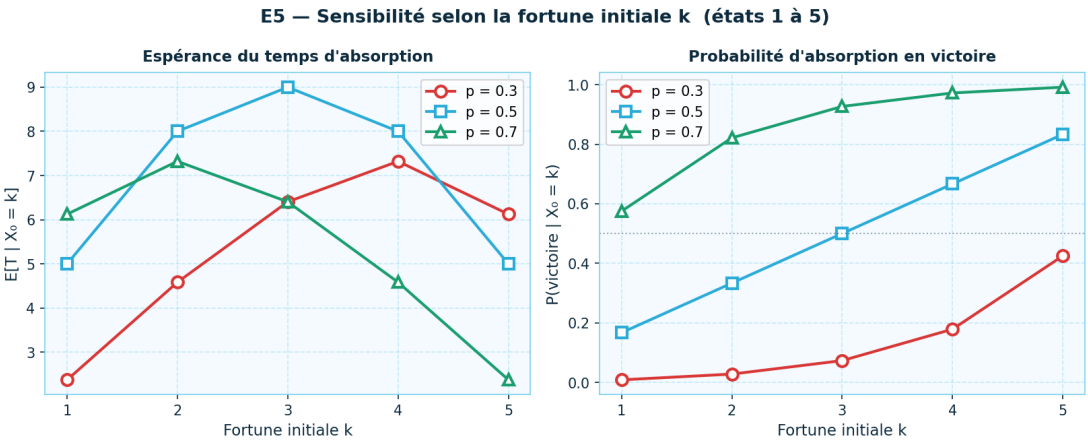

Figure 11: E5 – Esperance $E[T \mid X_0 = k]$ (gauche) et probabilite de victoire $P(\text{victoire} \mid X_0 = k)$ (droite) pour $p \in \{0.3, 0.5, 0.7\}$. La courbe $p = 0.5$ de P(victoire) est parfaitement lineaire $k/6$, conformement a la theorie.

Analyse qualitative :

- Pour $p = 0.5$: $P(\text{victoire} \mid k) = k/6$ (lineaire), $E[T \mid k]$ est maximal en $k = 3$ (etat le plus incertain).
- Pour $p > 0.5$: la courbe P(victoire) est convexe vers le haut — meme une fortune initiale faible offre une probabilite significative de victoire.
- Pour $p < 0.5$: la courbe est concave — la ruine est quasi-certaine sauf pour les fortunes tres proches de 6.
- $E[T]$ est toujours maximal a l'etat median $k = 3$: c'est l'etat qui maximise l'incertitude sur l'issue finale.

7 Discussion critique et Conclusion

7.1 Forces, limites et pistes d'amelioration

Forces

- A* optimal (h admissible), 55% d'expansions en moins vs UCS (17/20 graphes)
- Weighted A* : garantie bornee $\leq w \cdot C^*$, utile temps-reel
- Markov : resultats exacts via $N = (I - Q)^{-1}$, valides Monte Carlo ($< 0.2\%$)
- Architecture Python modulaire, reproductible, instrumentalisee

Limites & Pistes

- N en $O(n^3)$: prohibitif pour grands espaces (grilles 100×100)
- A* memoire lineaire : remplacer par IDA* ou SMA* si contrainte memoire
- Graphe statique : D* Lite ou ARA* pour environnements dynamiques
- Pistes : Bidirectional A*, Neural A*, MDP complet (equation de Bellman)

7.2 Conclusion

Ce mini-projet a demontre la complementarite entre recherche heuristique deterministe et modelisation stochastique.

Recherche heuristique : A* avec heuristique admissible et coherente garantit l'optimalite tout en reduisant l'exploration de 55% par rapport a UCS sur les grands graphes. Weighted A* offre un compromis w configurable entre vitesse et optimalite. La coherence de h supprime la necessite de re-ouvrir les noeuds de CLOSED.

Chaines de Markov absorbantes : La matrice fondamentale $N = (I - Q)^{-1}$ fournit des resultats analytiques exacts : pour un jeu equitable ($p = 0.5$), un joueur debutant en etat 2 a $\frac{2}{3}$ de chances de se ruiner en $E[T] = 8$ etapes – valide par Monte Carlo (200 000 essais, erreur $< 0.2\%$). Un leger desavantage ($p = 0.4$) accroit dramatiquement la ruine ($66.7\% \rightarrow 87.97\%$). L'extension E5 revele une sensibilite non lineaire de $E[T \mid k]$ selon la fortune initiale et la valeur de p.

Ces resultats ouvrent la voie a une integration dans un cadre MDP pour une planification robuste sous incertitude.

References

[1] Puterman, M.L. (1994). *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley.

[2] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

[3] Hart, P.E., Nilsson, N.J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Trans. SSC*, 4(2), 100–107.

[4] Pohl, I. (1970). Heuristic search viewed as path finding in a graph. *Artificial Intelligence*, 1(3-4), 193–204.

[5] Kemeny, J.G., & Snell, J.L. (1976). *Finite Markov Chains*. Springer.

[6] Norris, J.R. (1997). *Markov Chains*. Cambridge University Press.

[7] Dijkstra, E.W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271.

[8] Likhachev, M., Gordon, G., & Thrun, S. (2004). ARA*: Anytime A* with provable bounds on sub-optimality. *NIPS*, 16.

[9] Harris, C.R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.